

Activity_Course 2 TikTok project lab

September 28, 2025

1 TikTok Project

Course 2 - Get Started with Python

Welcome to the TikTok Project!

You have just started as a data professional at TikTok.

The team is still in the early stages of the project. You have received notice that TikTok's leadership team has approved the project proposal. To gain clear insights to prepare for a claims classification model, TikTok's provided data must be examined to begin the process of exploratory data analysis (EDA).

A notebook was structured and prepared to help you in this project. Please complete the following questions.

2 Course 2 End-of-course project: Inspect and analyze data

In this activity, you will examine data provided and prepare it for analysis.

The purpose of this project is to investigate and understand the data provided. This activity will:

1. Acquaint you with the data
2. Compile summary information about the data
3. Begin the process of EDA and reveal insights contained in the data
4. Prepare you for more in-depth EDA, hypothesis testing, and statistical analysis

The goal is to construct a dataframe in Python, perform a cursory inspection of the provided dataset, and inform TikTok data team members of your findings. *This activity has three parts:*

Part 1: Understand the situation * How can you best prepare to understand and organize the provided TikTok information?

Part 2: Understand the data

- Create a pandas dataframe for data learning and future exploratory data analysis (EDA) and statistical activities
- Compile summary information about the data to inform next steps

Part 3: Understand the variables

- Use insights from your examination of the summary data to guide deeper investigation into variables

To complete the activity, follow the instructions and answer the questions below. Then, you will use your responses to these questions and the questions included in the Course 2 PACE Strategy Document to create an executive summary.

Be sure to complete this activity before moving on to Course 3. You can assess your work by comparing the results to a completed exemplar after completing the end-of-course project.

3 Identify data types and compile summary information

Throughout these project notebooks, you'll see references to the problem-solving framework PACE. The following notebook components are labeled with the respective PACE stage: Plan, Analyze, Construct, and Execute.

4 PACE stages

- [Plan] (#scrollTo=psz51YkZVwtN&line=3&uniquifier=1)
- [Analyze] (#scrollTo=mA7Mz_SnI8km&line=4&uniquifier=1)
- [Construct] (#scrollTo=Lca9c8XON8lc&line=2&uniquifier=1)
- [Execute] (#scrollTo=401PgchTPr4E&line=2&uniquifier=1)

4.1 PACE: Plan

Consider the questions in your PACE Strategy Document and those below to craft your response:

4.1.1 Task 1. Understand the situation

- How can you best prepare to understand and organize the provided information?

Begin by exploring your dataset and consider reviewing the Data Dictionary.

To prepare, we would explore the dataset using `.head()`, `.info()`, and `.describe()` to get a sense of the structure and values. We'd also review the data dictionary to clearly understand what each column represents and how they relate to the classification goal.

- What follow-along and self-review codebooks will help you perform this work?

The most helpful codebooks are the ones that cover data cleaning, identifying null values, converting data types, and creating visualizations. These will guide me through organizing the data and preparing it for analysis.

- What are some additional activities a resourceful learner would perform before starting to code?

One would study the dataset and data dictionary carefully, outline a plan for cleaning and organizing the data, think about which features might be useful for the model, and consider the overall goal of classifying claims versus opinions to stay focused.

4.2 PACE: Analyze

Consider the questions in your PACE Strategy Document to reflect on the Analyze stage.

4.2.1 Task 2a. Imports and data loading

Start by importing the packages that you will need to load and explore the dataset. Make sure to use the following import statements: `* import pandas as pd`

- `import numpy as np`

```
[1]: # Import packages
import pandas as pd
import numpy as np
```

Then, load the dataset into a dataframe. Creating a dataframe will help you conduct data manipulation, exploratory data analysis (EDA), and statistical activities.

Note: As shown in this cell, the dataset has been automatically loaded in for you. You do not need to download the .csv file, or provide more code, in order to access the dataset and proceed with this lab. Please continue with this activity by completing the following instructions.

```
[2]: # Load dataset into dataframe
data = pd.read_csv("tiktok_dataset.csv")
```

4.2.2 Task 2b. Understand the data - Inspect the data

View and inspect summary information about the dataframe by **coding the following**:

1. `data.head(10)`
2. `data.info()`
3. `data.describe()`

Consider the following questions:

Question 1: When reviewing the first few rows of the dataframe, what do you observe about the data? What does each row represent?

Question 2: When reviewing the `data.info()` output, what do you notice about the different variables? Are there any null values? Are all of the variables numeric? Does anything else stand out?

Question 3: When reviewing the `data.describe()` output, what do you notice about the distributions of each variable? Are there any questionable values? Does it seem that there are outlier values?

```
[3]: # Display and examine the first ten rows of the dataframe
data.head(10)
```

```
[3]:
```

	#	claim_status	video_id	video_duration_sec	\
0	1	claim	7017666017	59	
1	2	claim	4014381136	32	
2	3	claim	9859838091	31	
3	4	claim	1866847991	25	
4	5	claim	7105231098	19	
5	6	claim	8972200955	35	
6	7	claim	4958886992	16	
7	8	claim	2270982263	41	
8	9	claim	5235769692	50	
9	10	claim	4660861094	45	

	video_transcription_text	verified_status	\
0	someone shared with me that drone deliveries a...	not verified	
1	someone shared with me that there are more mic...	not verified	
2	someone shared with me that american industria...	not verified	
3	someone shared with me that the metro of st. p...	not verified	
4	someone shared with me that the number of busi...	not verified	
5	someone shared with me that gross domestic pro...	not verified	
6	someone shared with me that elvis presley has ...	not verified	
7	someone shared with me that the best selling s...	not verified	
8	someone shared with me that about half of the ...	not verified	
9	someone shared with me that it would take a 50...	verified	

	author_ban_status	video_view_count	video_like_count	video_share_count	\
0	under review	343296.0	19425.0	241.0	
1	active	140877.0	77355.0	19034.0	
2	active	902185.0	97690.0	2858.0	
3	active	437506.0	239954.0	34812.0	
4	active	56167.0	34987.0	4110.0	
5	under review	336647.0	175546.0	62303.0	
6	active	750345.0	486192.0	193911.0	
7	active	547532.0	1072.0	50.0	
8	active	24819.0	10160.0	1050.0	
9	active	931587.0	171051.0	67739.0	

	video_download_count	video_comment_count
0	1.0	0.0
1	1161.0	684.0
2	833.0	329.0
3	1234.0	584.0
4	547.0	152.0
5	4293.0	1857.0
6	8616.0	5446.0

```

7          22.0          11.0
8          53.0          27.0
9         4104.0         2540.0

```

```
[4]: # Get summary info
data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19382 entries, 0 to 19381
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   #                     19382 non-null  int64
 1   claim_status          19084 non-null  object
 2   video_id              19382 non-null  int64
 3   video_duration_sec    19382 non-null  int64
 4   video_transcription_text 19084 non-null  object
 5   verified_status       19382 non-null  object
 6   author_ban_status     19382 non-null  object
 7   video_view_count      19084 non-null  float64
 8   video_like_count      19084 non-null  float64
 9   video_share_count     19084 non-null  float64
10   video_download_count   19084 non-null  float64
11   video_comment_count    19084 non-null  float64
dtypes: float64(5), int64(3), object(4)
memory usage: 1.8+ MB

```

```
[5]: summary = pd.DataFrame({
    'Column Name': data.columns,
    'Non-Null Count': data.notnull().sum().values,
    'Null Count': data.isnull().sum().values,
    'Data Type' : data.dtypes.values,
    '% Missing' : ((data.isnull().sum().values / len(data)) * 100).round(2)
})
summary
```

```
[5]:
```

	Column Name	Non-Null Count	Null Count	Data Type	% Missing
0	#	19382	0	int64	0.00
1	claim_status	19084	298	object	1.54
2	video_id	19382	0	int64	0.00
3	video_duration_sec	19382	0	int64	0.00
4	video_transcription_text	19084	298	object	1.54
5	verified_status	19382	0	object	0.00
6	author_ban_status	19382	0	object	0.00
7	video_view_count	19084	298	float64	1.54
8	video_like_count	19084	298	float64	1.54
9	video_share_count	19084	298	float64	1.54

10	video_download_count	19084	298	float64	1.54
11	video_comment_count	19084	298	float64	1.54

```
[6]: # Get summary statistics
data.describe()
```

```
[6]:
```

	#	video_id	video_duration_sec	video_view_count	\
count	19382.000000	1.938200e+04	19382.000000	19084.000000	
mean	9691.500000	5.627454e+09	32.421732	254708.558688	
std	5595.245794	2.536440e+09	16.229967	322893.280814	
min	1.000000	1.234959e+09	5.000000	20.000000	
25%	4846.250000	3.430417e+09	18.000000	4942.500000	
50%	9691.500000	5.618664e+09	32.000000	9954.500000	
75%	14536.750000	7.843960e+09	47.000000	504327.000000	
max	19382.000000	9.999873e+09	60.000000	999817.000000	

	video_like_count	video_share_count	video_download_count	\
count	19084.000000	19084.000000	19084.000000	
mean	84304.636030	16735.248323	1049.429627	
std	133420.546814	32036.174350	2004.299894	
min	0.000000	0.000000	0.000000	
25%	810.750000	115.000000	7.000000	
50%	3403.500000	717.000000	46.000000	
75%	125020.000000	18222.000000	1156.250000	
max	657830.000000	256130.000000	14994.000000	

	video_comment_count
count	19084.000000
mean	349.312146
std	799.638865
min	0.000000
25%	1.000000
50%	9.000000
75%	292.000000
max	9599.000000

RESPONSE TO QUESTIONS 1-3 HERE

Answer 1: Each row in the dataframe represents a single published TikTok video that has been labeled as either a claim or an opinion. The data includes various attributes about each video, such as its duration, transcription text, author status (e.g., verified or not), and engagement metrics like views, likes, shares, downloads, and comments. Most of the sample rows appear to be labeled as claims and come from unverified users.

Answer 2: The dataset contains **19382 rows and 12 columns**. Columns `claim_status`, `video_transcription_text`, `video_view_count`, `video_like_count`, `video_share_count`, `video_download_count`, and `video_comment_count` have **298 null values each**. Most columns are numeric (float64/int64), the rest are object like categorical columns `claim_status`,

verified_status and author_ban_status, and a string column video_transcription_text. There is a column named # which probably acts as a row ID.

Answer 3: Normal values for video_duration_sec which ranges from 5 to 60 seconds, as expected for TikTok contents. While the video view counts range from just 20 views to almost 1 million, this isn't a problem—it's actually normal for TikTok. A few videos go viral or trending, while most don't get as much attention. This pattern is common on social media and is not considered an error or outlier in a bad way. It's just how platforms like TikTok work. **No clearly invalid values** (like negative counts) are visible, but the skewness suggests we may want to apply transformations or handle outliers during analysis.

Initial Cleaning

- Replace column name # with row_id
- Converting claim_status, verified_status, and author_ban_status to category:
 - Saves memory
 - Speeds up operations like grouping, filtering
 - Makes the meaning of the data clearer (they're not meant for math)
- Converting video_transcription_text to string

```
[7]: tiktok_videos_df = data.rename(columns={'#': 'row_id'})
tiktok_videos_df['claim_status'] = tiktok_videos_df['claim_status'].
    ↳astype('category')
tiktok_videos_df['verified_status'] = tiktok_videos_df['verified_status'].
    ↳astype('category')
tiktok_videos_df['author_ban_status'] = tiktok_videos_df['author_ban_status'].
    ↳astype('category')
tiktok_videos_df['video_transcription_text'] =_
    ↳tiktok_videos_df['video_transcription_text'].astype('string')
tiktok_videos_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 19382 entries, 0 to 19381
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	row_id	19382 non-null	int64
1	claim_status	19084 non-null	category
2	video_id	19382 non-null	int64
3	video_duration_sec	19382 non-null	int64
4	video_transcription_text	19084 non-null	string
5	verified_status	19382 non-null	category
6	author_ban_status	19382 non-null	category
7	video_view_count	19084 non-null	float64
8	video_like_count	19084 non-null	float64
9	video_share_count	19084 non-null	float64
10	video_download_count	19084 non-null	float64
11	video_comment_count	19084 non-null	float64

```
dtypes: category(3), float64(5), int64(3), string(1)
memory usage: 1.4 MB
```

4.2.3 Task 2c. Understand the data - Investigate the variables

In this phase, you will begin to investigate the variables more closely to better understand them.

You know from the project proposal that the ultimate objective is to use machine learning to classify videos as either claims or opinions. A good first step towards understanding the data might therefore be examining the `claim_status` variable. Begin by determining how many videos there are for each different claim status.

```
[8]: # What are the different values for claim status and how many of each are in
      ↳ the data?
      tiktok_videos_df[['claim_status']].value_counts()
```

```
[8]: claim_status
      claim          9608
      opinion         9476
      dtype: int64
```

```
[9]: counts = tiktok_videos_df['claim_status'].value_counts()
      percentages = tiktok_videos_df['claim_status'].value_counts(normalize=True) *
      ↳ 100

      # Combine into a single DataFrame
      video_type_summary = pd.DataFrame({
          'count': counts,
          'percentage': percentages.round(2)
      })
      video_type_summary
```

```
[9]:          count  percentage
      claim     9608         50.35
      opinion    9476         49.65
```

Question: What do you notice about the values shown?

Answer: The number of values for both statuses is quite similar.

Next, examine the engagement trends associated with each different claim status.

Start by using Boolean masking to filter the data according to claim status, then calculate the mean and median view counts for each claim status.

```
[10]: # What is the average view count of videos with "claim" status?
      claim_videos = tiktok_videos_df[tiktok_videos_df['claim_status'] == 'claim']
      print(f"Mean view count of claim videos: {claim_videos['video_view_count'].
      ↳ mean()}")
```



```
print(f"Median view count of claim videos: {claim_videos['video_view_count'].
↳median()}")
```

Mean view count of claim videos: 501029.4527477102

Median view count of claim videos: 501555.0

```
[11]: opinion_videos = tiktok_videos_df[tiktok_videos_df['claim_status'] == 'opinion']
print(f"Mean view count of opinion videos: {opinion_videos['video_view_count'].
↳mean()}")
print(f"Median view count of opinion videos:↳
↳{opinion_videos['video_view_count'].median()}")
```

Mean view count of opinion videos: 4956.43224989447

Median view count of opinion videos: 4953.0

Question: What do you notice about the mean and media within each claim category?

Answer: For both claim and opinion statuses, the mean and median values are nearly the same, but opinion videos have lower view counts compared to claim videos.

Now, examine trends associated with the ban status of the author.

Use `groupby()` to calculate how many videos there are for each combination of categories of claim status and author ban status.

```
[12]: # Get counts for each group combination of claim status and author ban status
tiktok_videos_df.groupby(['claim_status', 'author_ban_status']).size()
```

```
[12]: claim_status  author_ban_status
claim            active          6566
              banned           1439
              under review        1603
opinion          active          8817
              banned            196
              under review        463
dtype: int64
```

Question: What do you notice about the number of claims videos with banned authors? Why might this relationship occur?

Answer: Even though **8817 active users** post **opinion videos**, these type of videos have only **196 banned users**. Compared to the **6566 active users** who submit **claim videos** which have **1439 banned users**. This suggests that users who make claim videos may be more likely to violate TikTok's policies, possibly because claim videos can include misleading, unverified, or harmful information. As a result, these users may face a higher chance of being banned.

Continue investigating engagement levels, now focusing on `author_ban_status`.

Calculate the median video share count of each author ban status.

```
[13]: ban_status_group = tiktok_videos_df.groupby(['author_ban_status'])
ban_status_group.agg({'video_view_count': ['mean', 'median', 'min', 'max'],
                      'video_like_count': ['mean', 'median', 'min', 'max'],
                      'video_share_count': ['mean', 'median', 'min', 'max']})
```

```
[13]:
```

	video_view_count				video_like_count \	
	mean	median	min	max	mean	
author_ban_status						
active	215927.039524	8616.0	20.0	999817.0	71036.533836	
banned	445845.439144	448201.0	72.0	997703.0	153017.236697	
under review	392204.836399	365245.5	75.0	999655.0	128718.050339	

	video_share_count \					
	median	min	max	mean	median	min
author_ban_status						
active	2222.0	0.0	657830.0	14111.466164	437.0	0.0
banned	105573.0	1.0	653561.0	29998.942508	14468.0	0.0
under review	71204.5	0.0	647236.0	25774.696999	9444.0	0.0

	max
author_ban_status	
active	256130.0
banned	249672.0
under review	238004.0

```
[14]: # What's the median video share count of each author ban status?
ban_status_group[['video_share_count']].median(numeric_only=True)
```

```
[14]:
```

	video_share_count
author_ban_status	
active	437.0
banned	14468.0
under review	9444.0

Question: What do you notice about the share count of banned authors, compared to that of active authors? Explore this in more depth.

Answer: Banned authors have much higher share counts than active ones. Their videos are shared more often, with a median of over **14468** compared to just **437** for active users. This may be because banned users post content that's more attention-grabbing or controversial, which goes viral quickly before getting removed.

Use `groupby()` to group the data by `author_ban_status`, then use `agg()` to get the count, mean, and median of each of the following columns: `* video_view_count * video_like_count * video_share_count`

Remember, the argument for the `agg()` function is a dictionary whose keys are columns. The values for each column are a list of the calculations you want to perform.

```
[15]: ban_status_group.agg({'video_view_count': ['count', 'mean', 'median', 'min', 'max'],
                             'video_like_count': ['count', 'mean', 'median', 'min', 'max'],
                             'video_share_count': ['count', 'mean', 'median', 'min', 'max']})
```

```
[15]:
```

	video_view_count \				
	count	mean	median	min	max
author_ban_status					
active	15383	215927.039524	8616.0	20.0	999817.0
banned	1635	445845.439144	448201.0	72.0	997703.0
under review	2066	392204.836399	365245.5	75.0	999655.0

	video_like_count \				
	count	mean	median	min	max
author_ban_status					
active	15383	71036.533836	2222.0	0.0	657830.0
banned	1635	153017.236697	105573.0	1.0	653561.0
under review	2066	128718.050339	71204.5	0.0	647236.0

	video_share_count				
	count	mean	median	min	max
author_ban_status					
active	15383	14111.466164	437.0	0.0	256130.0
banned	1635	29998.942508	14468.0	0.0	249672.0
under review	2066	25774.696999	9444.0	0.0	238004.0

Question: What do you notice about the number of views, likes, and shares for banned authors compared to active authors?

Answer: Banned authors have fewer videos overall, but their videos get much more attention. On average and at the median, their view, like, and share counts are much higher than those of active users. This shows that even though banned users post less, their content tends to go more viral. For each status, the mean values are higher than median. This is probably due to a few videos having very high engagement, which pulls the average up.

Now, create three new columns to help better understand engagement rates: * **likes_per_view**: represents the number of likes divided by the number of views for each video * **comments_per_view**: represents the number of comments divided by the number of views for each video * **shares_per_view**: represents the number of shares divided by the number of views for each video

```
[16]: # Create a likes_per_view column
tiktok_videos_df['likes_per_view'] = tiktok_videos_df['video_like_count'] /
    tiktok_videos_df['video_view_count']

# Create a comments_per_view column
```

```

tiktok_videos_df['comments_per_view'] = tiktok_videos_df['video_comment_count'] /
↳ tiktok_videos_df['video_view_count']

# Create a shares_per_view column
tiktok_videos_df['shares_per_view'] = tiktok_videos_df['video_share_count'] /
↳ tiktok_videos_df['video_view_count']

tiktok_videos_df.head(10)

```

```

[16]:
  row_id claim_status  video_id  video_duration_sec \
0      1      claim  7017666017          59
1      2      claim  4014381136          32
2      3      claim  9859838091          31
3      4      claim  1866847991          25
4      5      claim  7105231098          19
5      6      claim  8972200955          35
6      7      claim  4958886992          16
7      8      claim  2270982263          41
8      9      claim  5235769692          50
9     10      claim  4660861094          45

                                video_transcription_text verified_status \
0  someone shared with me that drone deliveries a...  not verified
1  someone shared with me that there are more mic...  not verified
2  someone shared with me that american industria...  not verified
3  someone shared with me that the metro of st. p...  not verified
4  someone shared with me that the number of busi...  not verified
5  someone shared with me that gross domestic pro...  not verified
6  someone shared with me that elvis presley has ...  not verified
7  someone shared with me that the best selling s...  not verified
8  someone shared with me that about half of the ...  not verified
9  someone shared with me that it would take a 50...  verified

  author_ban_status  video_view_count  video_like_count  video_share_count \
0      under review      343296.0      19425.0      241.0
1          active      140877.0      77355.0      19034.0
2          active      902185.0      97690.0      2858.0
3          active      437506.0     239954.0     34812.0
4          active       56167.0      34987.0      4110.0
5      under review      336647.0     175546.0     62303.0
6          active      750345.0     486192.0     193911.0
7          active      547532.0       1072.0        50.0
8          active       24819.0       10160.0       1050.0
9          active      931587.0     171051.0     67739.0

  video_download_count  video_comment_count  likes_per_view \
0              1.0              0.0      0.056584

```

1	1161.0	684.0	0.549096
2	833.0	329.0	0.108282
3	1234.0	584.0	0.548459
4	547.0	152.0	0.622910
5	4293.0	1857.0	0.521454
6	8616.0	5446.0	0.647958
7	22.0	11.0	0.001958
8	53.0	27.0	0.409364
9	4104.0	2540.0	0.183612

	comments_per_view	shares_per_view
0	0.000000	0.000702
1	0.004855	0.135111
2	0.000365	0.003168
3	0.001335	0.079569
4	0.002706	0.073175
5	0.005516	0.185069
6	0.007258	0.258429
7	0.000020	0.000091
8	0.001088	0.042306
9	0.002727	0.072714

Use `groupby()` to compile the information in each of the three newly created columns for each combination of categories of claim status and author ban status, then use `agg()` to calculate the count, the mean, and the median of each group.

```
[17]: claim_ban_status_group = tiktok_videos_df.groupby(['claim_status',
↳ 'author_ban_status'])
claim_ban_engagement_summary_df = claim_ban_status_group.agg({'likes_per_view':
↳ ['mean', 'median', 'min', 'max'],
↳
↳ 'comments_per_view': ['mean', 'median', 'min', 'max'],
↳ 'shares_per_view':
↳ ['mean', 'median', 'min', 'max']})
claim_ban_engagement_summary_df
```

```
[17]:
```

		likes_per_view \			
		mean	median	min	max
claim_status	author_ban_status				
claim	active	0.329542	0.326538	0.000000	0.666648
	banned	0.345071	0.358909	0.000259	0.666116
	under review	0.327997	0.320867	0.000815	0.665468
opinion	active	0.219744	0.218330	0.000000	0.444409
	banned	0.206868	0.198483	0.001802	0.444303
	under review	0.226394	0.228051	0.000000	0.443316

```

comments_per_view \

```

		mean	median	min	max
claim_status	author_ban_status				
claim	active	0.001393	0.000776	0.0	0.010280
	banned	0.001377	0.000746	0.0	0.009162
	under review	0.001367	0.000789	0.0	0.010049
opinion	active	0.000517	0.000252	0.0	0.004673
	banned	0.000434	0.000193	0.0	0.003032
	under review	0.000536	0.000293	0.0	0.003552

		shares_per_view			
		mean	median	min	max
claim_status	author_ban_status				
claim	active	0.065456	0.049279	0.000000	0.265956
	banned	0.067893	0.051606	0.000023	0.255618
	under review	0.065733	0.049967	0.000015	0.257914
opinion	active	0.043729	0.032405	0.000000	0.175632
	banned	0.040531	0.030728	0.000000	0.156641
	under review	0.044472	0.035027	0.000000	0.164691

```
[18]: # Add count separately using one column to represent all (they should all be
      ↳ the same)
      # Align the index before assigning
      count_series = claim_ban_status_group['likes_per_view'].count()
      claim_ban_engagement_summary_df.insert(0, 'count', count_series)

      # Flatten column names
      claim_ban_engagement_summary_df.columns = [
          column[0] if column[1] == '' else f'{column[0]}_{column[1]}'
          for column in claim_ban_engagement_summary_df.columns
      ]

      # Reset index to turn group keys into columns for a clean dataframe
      claim_ban_engagement_summary_df = claim_ban_engagement_summary_df.reset_index()
      claim_ban_engagement_summary_df
```

```
[18]: claim_status  author_ban_status  count  likes_per_view_mean  \
0      claim      active      6566      0.329542
1      claim      banned      1439      0.345071
2      claim      under review    1603      0.327997
3      opinion      active      8817      0.219744
4      opinion      banned       196      0.206868
5      opinion      under review    463      0.226394

      likes_per_view_median  likes_per_view_min  likes_per_view_max  \
0      0.326538      0.000000      0.666648
1      0.358909      0.000259      0.666116
2      0.320867      0.000815      0.665468
```

3	0.218330	0.000000	0.444409
4	0.198483	0.001802	0.444303
5	0.228051	0.000000	0.443316

	comments_per_view_mean	comments_per_view_median	comments_per_view_min \
0	0.001393	0.000776	0.0
1	0.001377	0.000746	0.0
2	0.001367	0.000789	0.0
3	0.000517	0.000252	0.0
4	0.000434	0.000193	0.0
5	0.000536	0.000293	0.0

	comments_per_view_max	shares_per_view_mean	shares_per_view_median \
0	0.010280	0.065456	0.049279
1	0.009162	0.067893	0.051606
2	0.010049	0.065733	0.049967
3	0.004673	0.043729	0.032405
4	0.003032	0.040531	0.030728
5	0.003552	0.044472	0.035027

	shares_per_view_min	shares_per_view_max
0	0.000000	0.265956
1	0.000023	0.255618
2	0.000015	0.257914
3	0.000000	0.175632
4	0.000000	0.156641
5	0.000000	0.164691

```
[19]: claim_verified_status_group = tiktok_videos_df.groupby(['claim_status',
↳ 'verified_status'])
claim_verified_engagement_summary_df = claim_verified_status_group.
↳ agg({'likes_per_view': ['mean', 'median', 'min', 'max'],
↳ 'comments_per_view': ['mean', 'median', 'min', 'max'],
↳ 'shares_per_view': ['mean', 'median', 'min', 'max']})
claim_verified_engagement_summary_df
```

```
[19]:
likes_per_view
mean median min max \
claim_status verified_status
claim not verified 0.331553 0.329708 0.000000 0.666648
verified 0.334156 0.337751 0.016313 0.659896
opinion not verified 0.219734 0.217545 0.000000 0.444409
verified 0.220386 0.220276 0.000523 0.444134
comments_per_view \
```

		mean	median	min	max
claim_status	verified_status				
claim	not verified	0.001382	0.000772	0.0	0.010280
	verified	0.001547	0.000893	0.0	0.008624
opinion	not verified	0.000519	0.000254	0.0	0.004673
	verified	0.000496	0.000232	0.0	0.003481

		shares_per_view			
		mean	median	min	max
claim_status	verified_status				
claim	not verified	0.065741	0.049459	0.000000	0.265956
	verified	0.071523	0.063413	0.000101	0.219662
opinion	not verified	0.043749	0.032384	0.000000	0.175632
	verified	0.043271	0.033088	0.000000	0.163709

Question: How does the data for claim videos and opinion videos compare or differ? Consider views, comments, likes, and shares.

Answer: Across all author statuses, claim videos generally have higher engagement metrics—views, likes, comments, and shares—compared to opinion videos. Earlier analysis suggested that banned authors tend to receive more engagement, but on closer look, their rates are only slightly higher than those of active and under-review authors. In fact, for opinion videos, banned authors actually show lower engagement rates. This suggests that engagement is not strongly linked to author status, but rather to the claim status of the content. Being banned is likely a result of the nature of the content itself, not the reason for its high engagement.

```
[20]: # Import vizualization packages
import seaborn as sns
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from matplotlib.ticker import PercentFormatter
from matplotlib.cbook import boxplot_stats
from IPython.display import display

# Formatter to display y-axis numbers with commas
formatter = FuncFormatter(lambda x, _: f'{int(x):,}')
```

```
# Set style
sns.set(style='whitegrid')
```

```
# Group data by claim status
tiktok_videos_status_group = tiktok_videos_df.groupby(['claim_status'])
```

```
[21]: # Helper function to safely add text if value is finite
def safe_text(ax, x, y, text, **kwargs):
    if np.isfinite(y):
```



```

        ax.text(x, y, f'{text}:\n{y:.2f}', va='center', fontsize=8, **kwargs)

def group_summary_stats(grouped_df, column_name):
    # Define the aggregation functions
    agg_funcs = {
        column_name: [lambda x: round(x.mean(), 2),
                       'median',
                       lambda x: x.quantile(0.25),
                       lambda x: x.quantile(0.75),
                       lambda x: x.quantile(0.75) - x.quantile(0.25),
                       'min',
                       'max']

    }

    # Perform the aggregation
    stats_df = grouped_df.agg(agg_funcs)
    stats_df.columns = ['mean', 'median', 'Q1', 'Q3', 'IQR', 'min', 'max']

    # Reset index for clean output
    stats_df = stats_df.reset_index()

    pd.set_option('display.max_columns', None)
    return stats_df

def group_summary_stats_multi(grouped_df, column_name_label_map):
    # Derive column names
    column_names = list(column_name_label_map.keys())

    # Loop through column and label mappings
    for column_name in column_names:
        print(f'{column_name_label_map[column_name]}')

        # Call group_summary_stats to generate statistical data
        stats_df = group_summary_stats(grouped_df, column_name)
        display(stats_df)

def plot_bar_by_group(source_df,
                      x_column_name,
                      x_column_label,
                      hue_column_name,
                      hue_column_label,
                      count_label):
    # Grouped data
    grouped_data = source_df.groupby([x_column_name, hue_column_name]).size().
    ↪reset_index(name='count')

    plt.figure(figsize=(10, 6))

```

```

    barplot = sns.barplot(data=grouped_data, x=x_column_name, y='count',
↪hue=hue_column_name)

    # Manually add text labels
    for bar in barplot.patches:
        height = bar.get_height()
        x = bar.get_x() + bar.get_width() / 2    # Center the text on the bar
        plt.text(x, height + 100, int(height), ha='center', va='bottom',
↪fontsize=9)

    plt.title(f'Distribution of {count_label} by {x_column_label} and
↪{hue_column_label}')
    plt.xlabel(f'{x_column_label}')
    plt.ylabel(f'{count_label}')
    plt.legend(title=hue_column_label)
    plt.tight_layout()
    plt.show()

def plot_box_by_group_multi(source_df,
                           column_name_label_map,
                           **kwargs):

    # Extract needed arguments from keyword arguments (kwargs)
    group_by_column_name = kwargs.get('group_by_column_name')
    grouped_df = kwargs.get('grouped_df')

    # Derive column names
    column_names = list(column_name_label_map.keys())

    box_count = len(column_name_label_map)
    chart_height = box_count * 6
    fig, axes = plt.subplots(box_count, 1, figsize=(10, chart_height))

    if box_count == 1:
        axes = [axes]    # Wrap single Axes into a list

    if grouped_df is None:
        grouped_df = source_df.groupby([group_by_column_name])

    # Loop through column and label mappings
    for i, column_name in enumerate(column_names):
        # fig, ax = plt.subplots(figsize=(10, 6))
        # Plot each box
        plot_box_by_group(source_df,
                           # ax
                           axes[i],
                           column_name,

```

```

        column_name_label_map[column_name],
        **kwargs)

plt.tight_layout()
plt.subplots_adjust(hspace=0.25)
plt.show()

def plot_box_by_group(source_df,
                      ax,
                      column_name,
                      column_label,
                      group_by_column_name,
                      group_by_column_label,
                      grouped_df=None,
                      show_legend=False):

    sns.boxplot(data=source_df, x=group_by_column_name, y=column_name, ax=ax)
    ax.yaxis.set_major_formatter(formatter)
    ax.set_title(f'Distribution of {column_label} by {group_by_column_label}')
    ax.set_xlabel(f'{group_by_column_label}')
    ax.set_ylabel(f'{column_label}')

    # Plot means
    means = grouped_df[column_name].mean()
    for i, mean in enumerate(means):
        if np.isfinite(mean): # Safeguard against NaN, inf, -inf
            ax.scatter(i, mean, color='red', marker='D',
                       s=40, zorder=5, label='Mean' if i == 0 else "")
            ax.hlines(mean, xmin=i - 0.4, xmax=i + 0.4,
                      colors='red', linestyle='dashed', linewidth=1)
            ax.text(i, mean, f'{mean:.2f}',
                    ha='center', va='bottom', fontsize=8)

    # Calculate boxplot stats
    stats = []
    for i, (name, group) in enumerate(grouped_df):
        stats = boxplot_stats(group[column_name])[0] # Get stats for this
        ↪ group

    # Min and Max (whiskers)
    safe_text(ax, i - 0.2, stats['whislo'], 'Lower Whisker', ha='right')
    safe_text(ax, i - 0.2, stats['whishi'], 'Upper Whisker', ha='right')

    # Q1 and Q3
    safe_text(ax, i + 0.2, stats['q1'], 'Q1', ha='left')
    safe_text(ax, i + 0.2, stats['q3'], 'Q3', ha='left')

    # Show legend only if there are labeled plot elements.

```

```

    # If no handles are found, it means all relevant values were NaN or
    →infinite,
    # so we print an info message instead of showing an empty legend.
    if show_legend:
        handles, labels = ax.get_legend_handles_labels()
        if handles:
            ax.legend()
        else:
            print(f'[INFO] Legend not shown for "{column_label}" because all
    →relevant values were NaN or infinite.')

def plot_box_with_stats_by_group(source_df,
                                column_name,
                                column_label,
                                group_by_column_name,
                                group_by_column_label,
                                grouped_df=None,
                                show_legend=False):
    fig, ax = plt.subplots(figsize=(10, 6))

    if grouped_df is None:
        grouped_df = source_df.groupby([group_by_column_name])

    # ***** Plot box chart *****
    plot_box_by_group(source_df,
                      ax,
                      column_name,
                      column_label,
                      group_by_column_name,
                      group_by_column_label,
                      grouped_df,
                      show_legend)

    plt.tight_layout()
    plt.show()

    # ***** Show statistical summary *****
    print(f'{column_label} per {group_by_column_label}')
    display(group_summary_stats(grouped_df, column_name))

def plot_histogram_all_and_by_group(source_df,
                                    x_column_name,
                                    x_column_label,
                                    group_by_column_name,
                                    group_by_column_label,
                                    count_label,
                                    grouped_df=None,
                                    as_percentage=False,

```

```

include_overall=True):

if include_overall:
    # ***** Plot histogram for overall data *****
    # print('\n')
    plt.figure(figsize=(12, 5))
    hist_overall = sns.histplot(source_df[x_column_name],
                                bins=50,
                                kde=True)
    plt.title(f'Overall Distribution of {x_column_label}')
    plt.xlabel(x_column_label)
    plt.ylabel(count_label)

    # Format x-axis as percentage (optional)
    if as_percentage:
        plt.gca().xaxis.set_major_formatter(PercentFormatter(xmax=1))

    # Rotate x-axis labels
    # plt.xticks(rotation=45)

    # Add value labels on top of bars
    for patch in hist_overall.patches:
        height = patch.get_height()
        if height > 0:
            plt.text(patch.get_x() + patch.get_width() / 2, height + 5,
↪f'{int(height)}',
                        ha='center', va='bottom', fontsize=8, rotation=45)

    plt.tight_layout()
    plt.show()

    # ***** Show overall statistical summary *****
    print(f'Overall {x_column_label}')
    summary_df = pd.DataFrame([
        {
            'count': source_df[x_column_name].count(),
            'mean': source_df[x_column_name].mean(),
            'median': source_df[x_column_name].median(),
            'min': source_df[x_column_name].min(),
            'max': source_df[x_column_name].max()
        }
    ])
    display(summary_df)
    print('\n')

    # ***** Plot histogram by group *****
    plt.figure(figsize=(12, 5))

```

```

hist_by_group = sns.histplot(data=source_df, x=x_column_name,
↪hue=group_by_column_name,
                                bins=50, kde=True, element='bars',
↪stat='count', common_norm=False)
plt.title(f'Distribution of {x_column_label} by {group_by_column_label}')
plt.xlabel(x_column_label)
plt.ylabel(count_label)

if hist_by_group.get_legend() is not None:
    hist_by_group.get_legend().set_title(group_by_column_label)

# Format x-axis as percentage (optional)
if as_percentage:
    plt.gca().xaxis.set_major_formatter(PercentFormatter(xmax=1))

# Rotate x-axis labels
# plt.xticks(rotation=45)

# Add value labels per hue group
for container in hist_by_group.containers:
    for bar in container:
        height = bar.get_height()
        if height > 0:
            hist_by_group.annotate(f'{int(height)}', xy=(bar.get_x() + bar.
↪get_width() / 2, height),
                                xytext=(0, 4), textcoords='offset_
↪points', ha='center',
                                va='bottom', fontsize=8, rotation=45)

plt.tight_layout()
plt.show()

# ***** Show statistical summary by group *****
if grouped_df is None:
    grouped_df = source_df.groupby([group_by_column_name])

print(f'{x_column_label} by {group_by_column_label}')
group_summary_df = grouped_df.agg({x_column_name: ['count', 'mean',
↪'median', 'min', 'max']})

# Flatten multi-level columns
group_summary_df.columns = [f'{stat}' for stat in group_summary_df.columns.
↪get_level_values(1)]
group_summary_df = group_summary_df.reset_index()
display(group_summary_df)

def plot_heatmap(source_df, column_name_label_map, title):

```

```
plt.figure(figsize=(10, 8))

# Derive column names
column_names = list(column_name_label_map.keys())

# Compute correlation and rename for display
correlation = source_df[column_names].corr()
correlation.rename(index=column_name_label_map,
→columns=column_name_label_map, inplace=True)

sns.heatmap(correlation, annot=True, cmap='coolwarm', fmt='.2f')
plt.title(title)
plt.show()
```

PACE: Analyze Stage Questions **Question 1:** Will the available information be sufficient to achieve the goal based on your intuition and the analysis of the variables?

Question 2: How would you build summary dataframe statistics and assess the min and max range of the data?

Question 3: Do the averages of any of the data variables look unusual? Can you describe the interval data?

Answer 1: Yes, the available information seems sufficient to support the early stages of building a claim classification model. The dataset includes labeled data (`claim_status`) and engagement metrics, along with user and content-level variables, which are useful for both exploratory analysis and feature development.

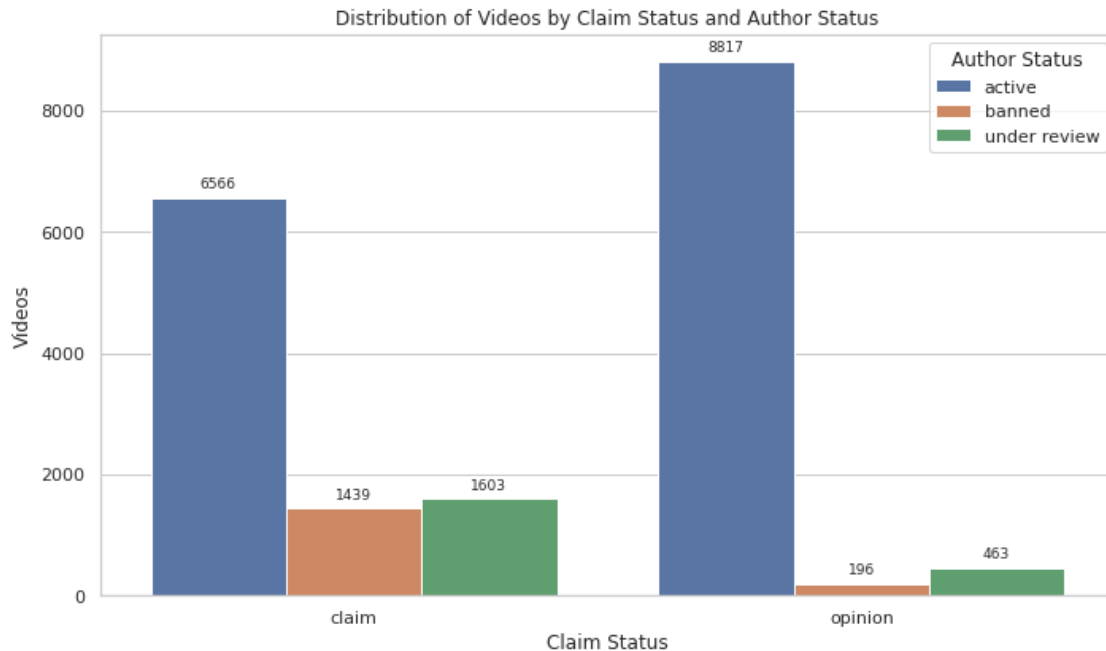
Answer 2: We grouped the dataset by `claim_status` and `author_ban_status`, then calculated mean, median, min, and max for the derived engagement columns: `likes_per_view`, `comments_per_view`, and `shares_per_view`. These columns were computed earlier from raw counts. We flattened the multi-level column names and reset the index to turn the result into a readable summary dataframe, making it easy to analyze how engagement varies across claim and ban status groups.

Answer 3: Some of the averages are noticeably higher than the medians, especially in the engagement metrics, which suggests skewness caused by a few high-performing (possibly viral or controversial) videos. The interval data reflects this, showing wide ranges—for example, `video_view_count` ranges from just 20 to nearly 1 million. Most videos have low view counts, but a small number receive very high engagement. These outliers create a wide range and pull the average up, making it higher than what most videos typically receive.

4.2.4 Visualization

Compare the count of claim vs. opinion videos by `author_ban_status`.

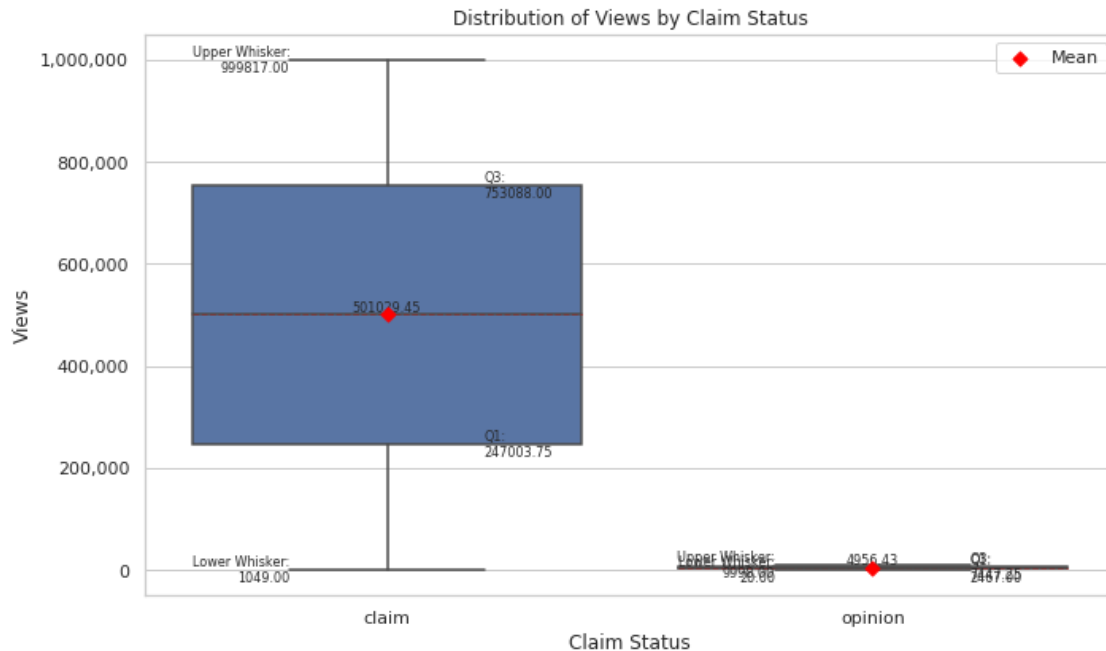
```
[22]: plot_bar_by_group(tiktok_videos_df, 'claim_status', 'Claim Status',
→'author_ban_status', 'Author Status', 'Videos')
```



The chart shows the number of videos categorized by both `claim_status` and `author_ban_status`. Claim videos with active authors account for a large portion, followed by smaller but similar numbers from banned and under review authors. In contrast, most opinion videos come from active authors, with significantly fewer from banned or under review accounts. This suggests that claim content is more frequently associated with authors who face moderation, while opinion content is largely produced by active users.

Show the distribution of engagement metrics—views, likes, shares, and comments—across `claim_status` to highlight outliers and differences in medians.

```
[23]: plot_box_with_stats_by_group(tiktok_videos_df, 'video_view_count', 'Views',
                                   'claim_status', 'Claim Status',
                                   tiktok_videos_status_group, True)
```

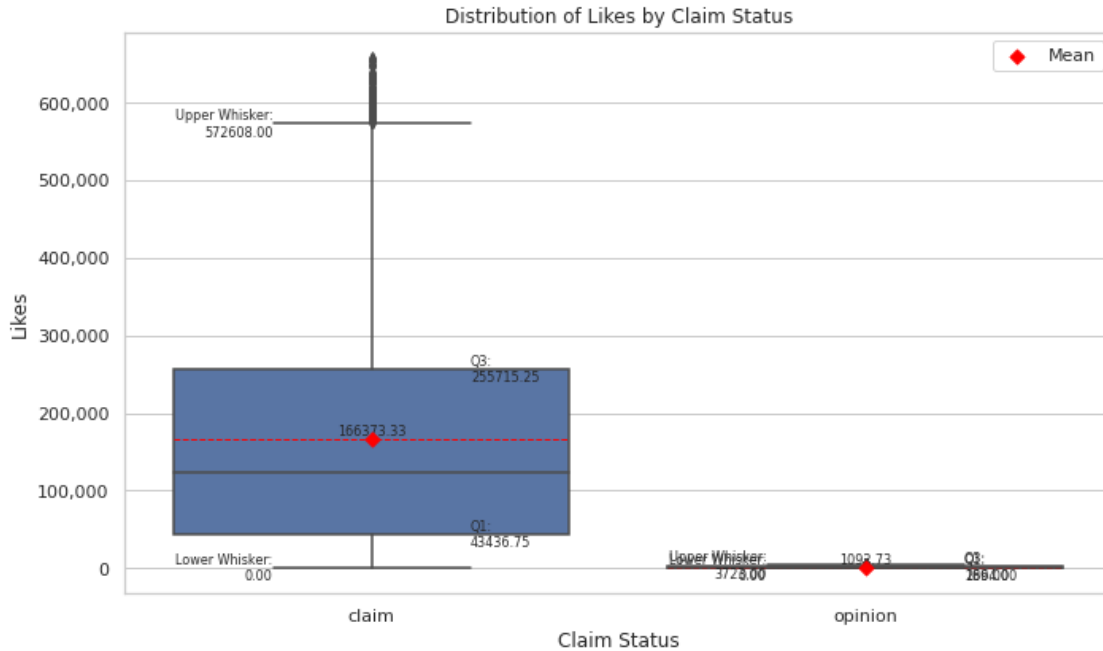



Views per Claim Status

	claim_status	mean	median	Q1	Q3	IQR	min \
0	claim	501029.45	501555.0	247003.75	753088.00	506084.25	1049.0
1	opinion	4956.43	4953.0	2467.00	7447.25	4980.25	20.0
	max						
0		999817.0					
1		9998.0					

The chart illustrates the distribution of video views by `claim_status`. Videos with a `claim` status have a significantly higher view count than `opinion` videos, as shown by the immense difference in scale and the respective summary statistics. The `median` view count for `claim` videos is 501,555, while for `opinion` videos it is only 4,953. This indicates that `claim` videos, on average, are viewed far more frequently than `opinion` videos.

```
[24]: plot_box_with_stats_by_group(tiktok_videos_df, 'video_like_count', 'Likes',
                                     'claim_status', 'Claim Status',
                                     tiktok_videos_status_group, True)
```



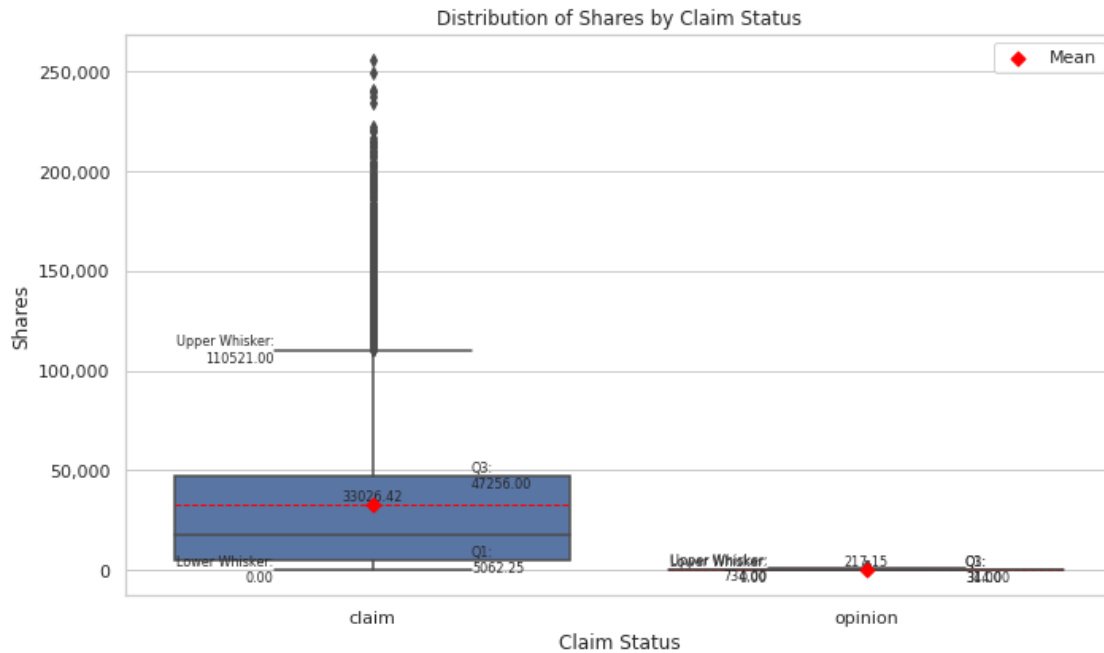
Likes per Claim Status

	claim_status	mean	median	Q1	Q3	IQR	min \
0	claim	166373.33	123649.0	43436.75	255715.25	212278.5	0.0
1	opinion	1092.73	823.0	289.00	1664.00	1375.0	0.0

	max
0	657830.0
1	4375.0

The chart displays the distribution of likes by `claim_status`. Similar to views, the number of likes for `claim` videos is substantially greater than for `opinion` videos. The `median` likes for `claim` videos are 123,649, whereas for `opinion` videos the `median` is 823. This suggests a strong correlation between video `claim_status` and the volume of likes received, with `claim` videos garnering significantly more positive engagement.

```
[25]: plot_box_with_stats_by_group(tiktok_videos_df, 'video_share_count', 'Shares',
                                   'claim_status', 'Claim Status',
                                   tiktok_videos_status_group, True)
```

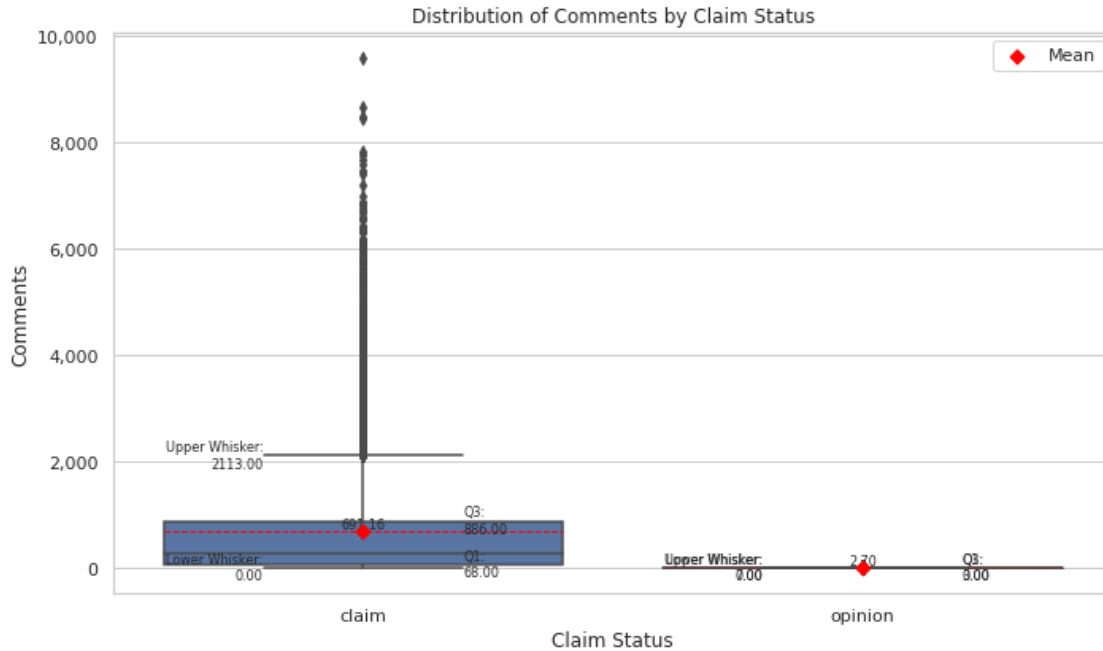


Shares per Claim Status

	claim_status	mean	median	Q1	Q3	IQR	min	max
0	claim	33026.42	17997.5	5062.25	47256.0	42193.75	0.0	256130.0
1	opinion	217.15	121.0	34.00	314.0	280.00	0.0	1674.0

The chart shows the distribution of shares by `claim_status`. The data reveals that `claim` videos are shared much more frequently than `opinion` videos. The median number of shares for `claim` videos is 17,997.5, which is far greater than the median of 121 for `opinion` videos. This indicates that `claim` videos have a higher potential for virality and reach through sharing compared to `opinion` videos.

```
[26]: plot_box_with_stats_by_group(tiktok_videos_df, 'video_comment_count',
    ↳ 'Comments',
    ↳ 'claim_status', 'Claim Status',
    ↳ tiktok_videos_status_group, True)
```



Comments per Claim Status

	claim_status	mean	median	Q1	Q3	IQR	min	max
0	claim	691.16	286.0	68.0	886.0	818.0	0.0	9599.0
1	opinion	2.70	1.0	0.0	3.0	3.0	0.0	32.0

The chart presents the distribution of comments by `claim_status`. The data shows a similar pattern to the other metrics, with `claim` videos receiving a much higher number of comments. The **median** comment count for `claim` videos is 286, while `opinion` videos have a **median** of only 1. This analysis suggests that `claim` videos are more effective at stimulating discussion and user engagement in the form of comments.

4.2.5 Data Analyst Notes

Retrieving Q1, Q3, IQR, Lower Whisker, and Upper Whisker Given observations:
[10, 20, 30, 40, 50]

* `n` = 5 number of observations * `min` = 10 * `max` = 50

Quartile Formula:

$$Q = \text{value_at_floor} + (\text{position} - \text{floor}) * (\text{value_at_ceil} - \text{value_at_floor})$$

Q1: 25th percentile (Lower Quartile)

$$\begin{aligned}
 Q1 \text{ position} &= (n + 1) * 0.25 \\
 &= (5 + 1) * 0.25 \\
 &= 1.5
 \end{aligned}$$

$$\begin{aligned}
 \text{floor index} &= 1 \\
 \text{ceil index} &= 2 \\
 \text{value at floor index} &= 10 \\
 \text{value at ceil index} &= 20
 \end{aligned}$$

$$\begin{aligned}
 Q1 &= 10 + (1.5 - 1) * (20 - 10) \\
 &= 10 + 0.5 * 10 \\
 &= 15
 \end{aligned}$$

Q3: 75th percentile (Upper Quartile)

$$\begin{aligned}
 Q3 \text{ position} &= (n + 1) * 0.75 \\
 &= (5 + 1) * 0.75 \\
 &= 4.5
 \end{aligned}$$

$$\begin{aligned}
 \text{floor index} &= 4 \\
 \text{ceil index} &= 5 \\
 \text{value at floor index} &= 40 \\
 \text{value at ceil index} &= 50
 \end{aligned}$$

$$\begin{aligned}
 Q3 &= 40 + (4.5 - 4) * (50 - 40) \\
 &= 40 + 0.5 * 10 \\
 &= 45
 \end{aligned}$$

IQR: Interquartile Range

$$\begin{aligned}
 IQR &= Q3 - Q1 \\
 &= 45 - 15 \\
 &= 30
 \end{aligned}$$

Whiskers Lower whisker

$$\begin{aligned}
 \text{lower_whisker} &= Q1 - 1.5 * IQR \\
 &= 15 - 1.5 * 30 \\
 &= 15 - 45 \\
 &= -30
 \end{aligned}$$

Actual minimum = 10

Since 10 > -30 → lower whisker is **10**

Upper whisker

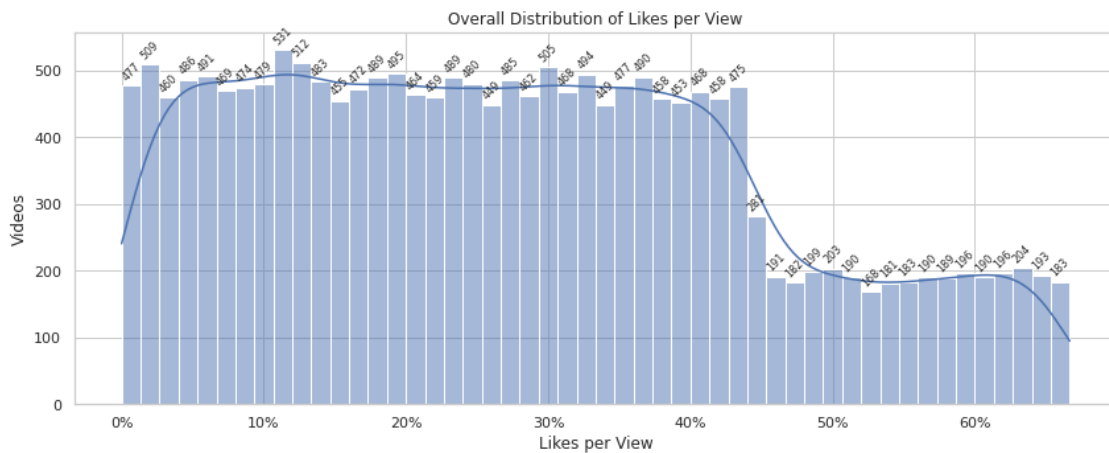
```
upper_whisker = Q3 + 1.5 * IQR
               = 45 + 1.5 * 30
               = 45 + 45
               = 90
```

Actual maximum = 50

Since 50 < 90 → upper whisker is **50**

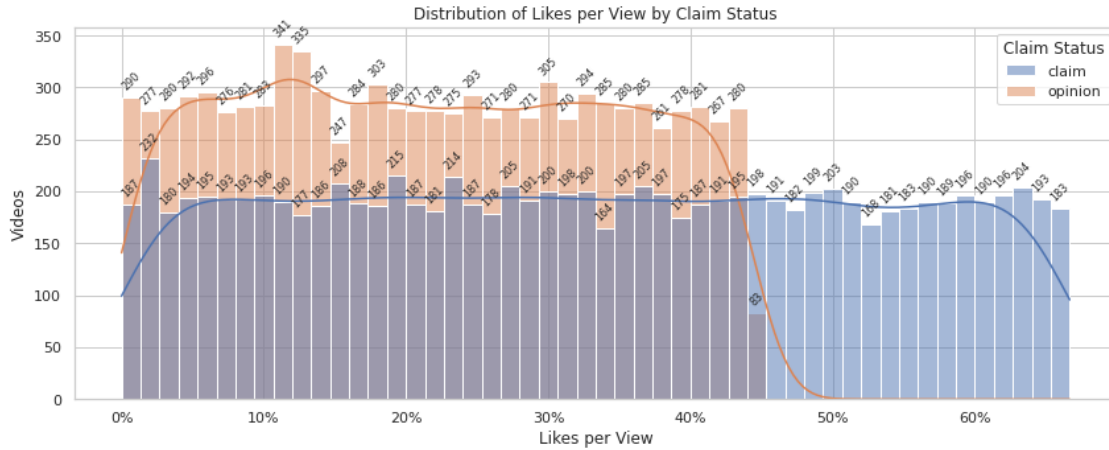
Visualize the distribution of likes, shares, and comments per view, overall and by claim_status, to assess whether engagement patterns can help distinguish claims from opinions.

```
[27]: plot_histogram_all_and_by_group(tiktok_videos_df, 'likes_per_view', 'Likes per View',
                                     'claim_status', 'Claim Status', 'Videos',
                                     tiktok_videos_status_group, True)
```



Overall Likes per View

	count	mean	median	min	max
0	19084	0.276093	0.264037	0.0	0.666648

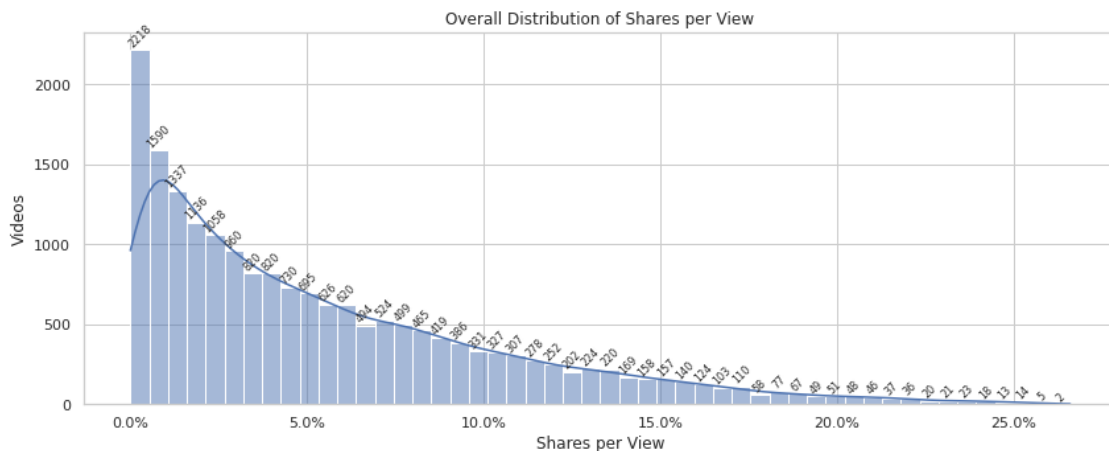


Likes per View by Claim Status

	claim_status	count	mean	median	min	max
0	claim	9608	0.331610	0.329734	0.0	0.666648
1	opinion	9476	0.219802	0.218127	0.0	0.444409

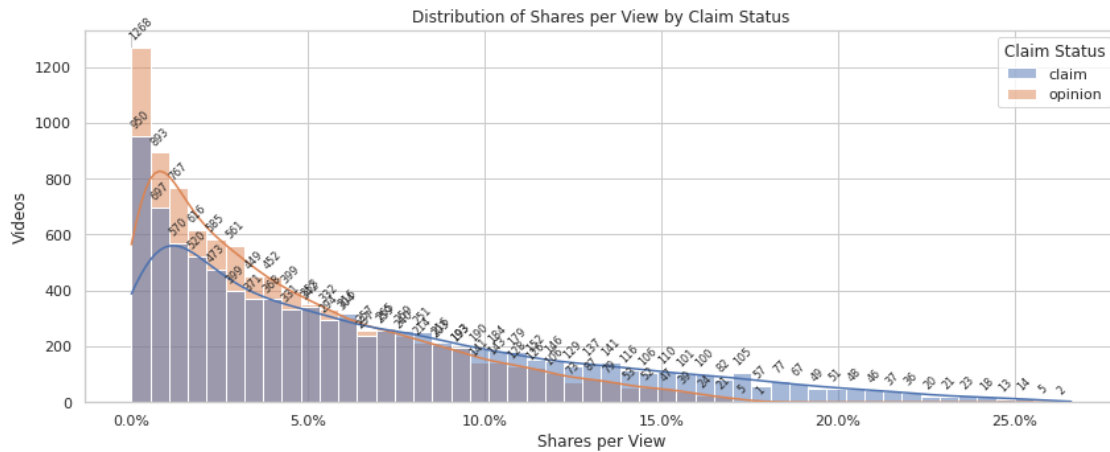
The charts show the distribution of likes per view, both overall and by `claim_status`. Overall, the distribution of videos is relatively even across different “Likes per View” percentages up to around 40%, after which there is a significant drop. Videos with a `claim` status are more successful at generating likes per view than `opinion` videos. This is evidenced by the higher `mean` (0.331610 vs. 0.219802) and `median` (0.329734 vs. 0.218127) for `claim` videos. The consistent difference in `mean` and `median` values highlights a clear advantage in user engagement for `claim` videos when it comes to likes.

```
[28]: plot_histogram_all_and_by_group(tiktok_videos_df, 'shares_per_view', 'Shares_
      ↳per View',
      'claim_status', 'Claim Status', 'Videos',
      ↳tiktok_videos_status_group, True)
```



Overall Shares per View

	count	mean	median	min	max
0	19084	0.05486	0.039739	0.0	0.265956

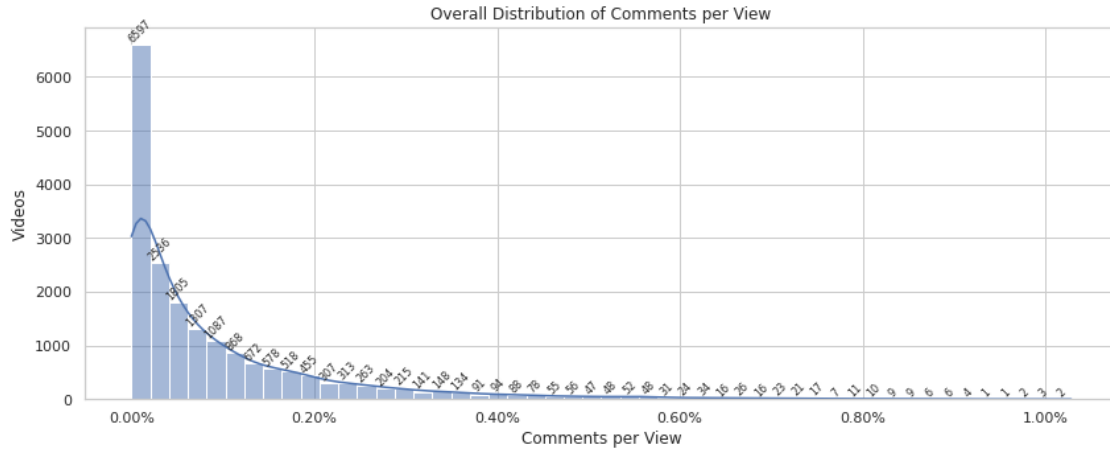


Shares per View by Claim Status

	claim_status	count	mean	median	min	max
0	claim	9608	0.065867	0.049673	0.0	0.265956
1	opinion	9476	0.043699	0.032487	0.0	0.175632

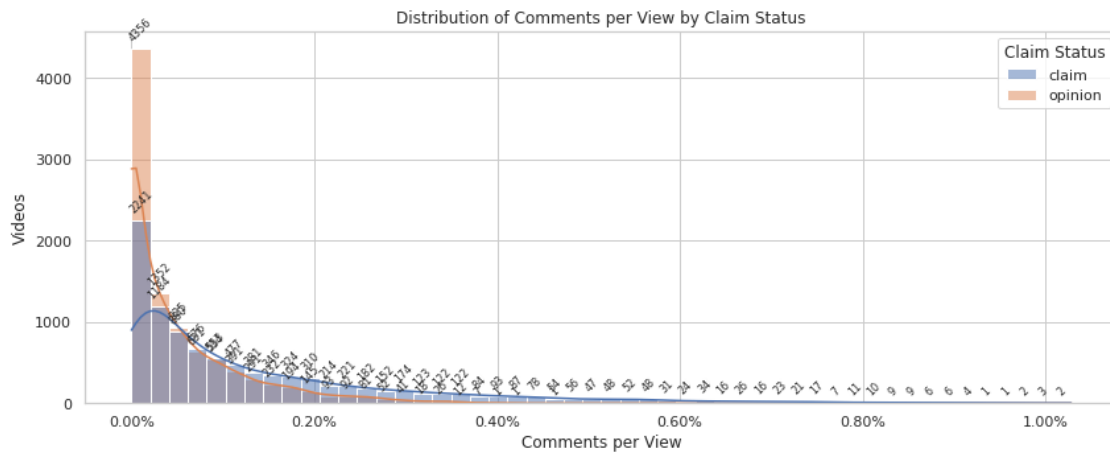
The charts present the distribution of `shares_per_view`. The overall distribution is heavily skewed towards the lower percentages, meaning most videos have a low “Shares per View” rate. Nevertheless, `claim` videos have a higher `mean` (0.065867) and `median` (0.049673) for `shares_per_view` than `opinion` videos, which have a `mean` of 0.043699 and a `median` of 0.032487. This reveals that sharing behavior is also more prevalent for `claim` videos.

```
[29]: plot_histogram_all_and_by_group(tiktok_videos_df, 'comments_per_view',
    ↳ 'Comments per View',
    ↳ 'claim_status', 'Claim Status', 'Videos',
    ↳ tiktok_videos_status_group, True)
```

Overall Comments per View

	count	mean	median	min	max
0	19084	0.000954	0.000455	0.0	0.01028



Comments per View by Claim Status

	claim_status	count	mean	median	min	max
0	claim	9608	0.001386	0.000774	0.0	0.010280
1	opinion	9476	0.000516	0.000253	0.0	0.004673

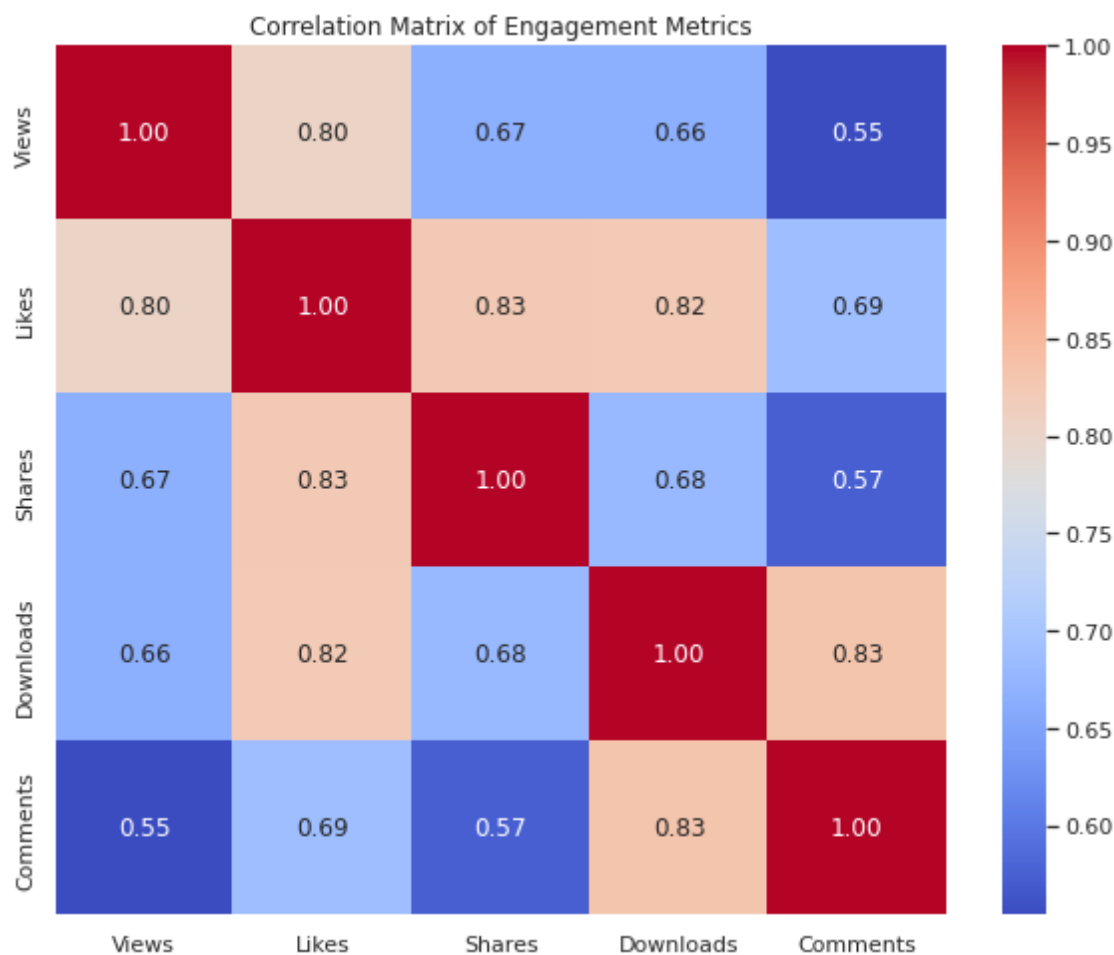
The charts illustrate the distribution of `comments_per_view`. Similar to shares, the overall distribution is heavily concentrated at the lower end, indicating that most videos have a very low

“Comments per View” rate. However, even within this low-engagement context, `claim` videos still generate a higher rate of comments. The `mean` “Comments per View” for `claim` videos is 0.001386, which is more than double the `mean` for `opinion` videos at 0.000516. Similarly, the `median` “Comments per View” is 0.000774 for `claim` videos and 0.000253 for `opinion` videos. This suggests that `claim` videos may be more effective at stimulating discussion and user feedback, despite the overall low volume of comments across all videos.

This pattern across all three engagement metrics—likes, comments, and shares—indicates that content with a `claim` status is consistently more effective at driving user interaction and virality.

Visualizes the relationships among five key engagement metrics: Views, Likes, Shares, Downloads, and Comments.

```
[30]: plot_heatmap(tiktok_videos_df,
                  {
                      'video_view_count': 'Views', 'video_like_count': 'Likes',
                      ↪ 'video_share_count': 'Shares',
                      'video_download_count': 'Downloads', 'video_comment_count':
                      ↪ 'Comments'
                  },
                  'Correlation Matrix of Engagement Metrics')
```



The heatmap shows how engagement metrics—views, likes, shares, downloads, and comments—relate to one another. Most of these metrics are strongly correlated, especially likes with shares and downloads. Views also show a strong relationship with likes but a weaker one with comments.

These results suggest that videos with higher engagement in one area tend to perform well in others. However, due to the high overlap, some metrics may be redundant. Creating ratios like `likes_per_view` or `shares_per_view` could help reduce noise and highlight meaningful differences in user interaction. While this analysis doesn't directly support claim classification, it helps inform which features to focus on when building the model.

4.3 PACE: Construct

Note: The Construct stage does not apply to this workflow. The PACE framework can be adapted to fit the specific requirements of any project.

4.4 PACE: Execute

Consider the questions in your PACE Strategy Document and those below to craft your response.

Question 1: Given your current knowledge of the data, what would you initially recommend to your manager to investigate further prior to performing exploratory data analysis?

Question 2: What data initially presents as containing anomalies?

Question 3: What additional types of data could strengthen this dataset?

Answer 1: To support the goal of building a model that identifies whether a video is a claim or an opinion, We recommend focusing on features that show clear differences between the two categories, such as engagement metrics—views, likes, shares, and comments. These patterns can help identify the most useful predictors for the model.

Answer 2: Engagement metrics such as `video_view_count`, `video_like_count`, and `video_share_count` show signs of skewness due to extreme values. These high outliers may distort analysis and should be flagged for further review.

Answer 3: Including data on the number of user reports per video, timestamps, user demographics, or content categories (e.g., politics, health, entertainment) could provide stronger context for classifying claims and understanding engagement patterns.

4.4.1 Given your efforts, what can you summarize for Rosie Mae Bradshaw and the TikTok data team?

Note for Learners: Your answer should address TikTok’s request for a summary that covers the following points:

- What percentage of the data is comprised of claims and what percentage is comprised of opinions?
- What factors correlate with a video’s claim status?
- What factors correlate with a video’s engagement level?
- From the total entries of **19382**, claims have a percentage of **50.35%** or **9608** entries, while opinions have **49.65%** or **9476** entries.
- Major factors are the engagement metrics—views, likes, comments, and shares.
-

4.5 Claim videos consistently receive higher engagement metrics across all author statuses. Although author ban status may slightly affect engagement, it seems less influential than claim status. So, the nature of the content, especially whether it’s a claim, appears to be the strongest factor linked to engagement.

```
[31]: tiktok_videos_df.head(10)
```

```
[31]:  row_id claim_status    video_id  video_duration_sec  \
0      1      claim  7017666017          59
1      2      claim  4014381136          32
2      3      claim  9859838091          31
3      4      claim  1866847991          25
4      5      claim  7105231098          19
5      6      claim  8972200955          35
6      7      claim  4958886992          16
7      8      claim  2270982263          41
8      9      claim  5235769692          50
9     10      claim  4660861094          45
```

```
      video_transcription_text verified_status  \
0  someone shared with me that drone deliveries a...  not verified
1  someone shared with me that there are more mic...  not verified
2  someone shared with me that american industria...  not verified
3  someone shared with me that the metro of st. p...  not verified
4  someone shared with me that the number of busi...  not verified
5  someone shared with me that gross domestic pro...  not verified
6  someone shared with me that elvis presley has ...  not verified
7  someone shared with me that the best selling s...  not verified
8  someone shared with me that about half of the ...  not verified
9  someone shared with me that it would take a 50...  verified
```

```
author_ban_status  video_view_count  video_like_count  video_share_count  \
0      under review      343296.0      19425.0      241.0
1      active      140877.0      77355.0      19034.0
2      active      902185.0      97690.0      2858.0
3      active      437506.0      239954.0      34812.0
4      active      56167.0      34987.0      4110.0
5      under review      336647.0      175546.0      62303.0
6      active      750345.0      486192.0      193911.0
7      active      547532.0      1072.0      50.0
8      active      24819.0      10160.0      1050.0
9      active      931587.0      171051.0      67739.0
```

```
video_download_count  video_comment_count  likes_per_view  \
0      1.0      0.0      0.056584
1      1161.0      684.0      0.549096
2      833.0      329.0      0.108282
3      1234.0      584.0      0.548459
4      547.0      152.0      0.622910
5      4293.0      1857.0      0.521454
6      8616.0      5446.0      0.647958
7      22.0      11.0      0.001958
8      53.0      27.0      0.409364
9      4104.0      2540.0      0.183612
```

	comments_per_view	shares_per_view
0	0.000000	0.000702
1	0.004855	0.135111
2	0.000365	0.003168
3	0.001335	0.079569
4	0.002706	0.073175
5	0.005516	0.185069
6	0.007258	0.258429
7	0.000020	0.000091
8	0.001088	0.042306
9	0.002727	0.072714

```
[32]: tiktok_videos_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19382 entries, 0 to 19381
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   row_id                                19382 non-null  int64
1   claim_status                          19084 non-null  category
2   video_id                              19382 non-null  int64
3   video_duration_sec                   19382 non-null  int64
4   video_transcription_text             19084 non-null  string
5   verified_status                      19382 non-null  category
6   author_ban_status                    19382 non-null  category
7   video_view_count                     19084 non-null  float64
8   video_like_count                     19084 non-null  float64
9   video_share_count                    19084 non-null  float64
10  video_download_count                 19084 non-null  float64
11  video_comment_count                  19084 non-null  float64
12  likes_per_view                       19084 non-null  float64
13  comments_per_view                    19084 non-null  float64
14  shares_per_view                      19084 non-null  float64
dtypes: category(3), float64(8), int64(3), string(1)
memory usage: 1.8 MB
```

```
[33]: tiktok_videos_df.describe()
```

```
[33]:
```

	row_id	video_id	video_duration_sec	video_view_count	\
count	19382.000000	1.938200e+04	19382.000000	19084.000000	
mean	9691.500000	5.627454e+09	32.421732	254708.558688	
std	5595.245794	2.536440e+09	16.229967	322893.280814	
min	1.000000	1.234959e+09	5.000000	20.000000	
25%	4846.250000	3.430417e+09	18.000000	4942.500000	
50%	9691.500000	5.618664e+09	32.000000	9954.500000	

75%	14536.750000	7.843960e+09	47.000000	504327.000000
max	19382.000000	9.999873e+09	60.000000	999817.000000

	video_like_count	video_share_count	video_download_count	\
count	19084.000000	19084.000000	19084.000000	
mean	84304.636030	16735.248323	1049.429627	
std	133420.546814	32036.174350	2004.299894	
min	0.000000	0.000000	0.000000	
25%	810.750000	115.000000	7.000000	
50%	3403.500000	717.000000	46.000000	
75%	125020.000000	18222.000000	1156.250000	
max	657830.000000	256130.000000	14994.000000	

	video_comment_count	likes_per_view	comments_per_view	shares_per_view
count	19084.000000	19084.000000	19084.000000	19084.000000
mean	349.312146	0.276093	0.000954	0.054860
std	799.638865	0.173006	0.001326	0.050597
min	0.000000	0.000000	0.000000	0.000000
25%	1.000000	0.130240	0.000098	0.014445
50%	9.000000	0.264037	0.000455	0.039739
75%	292.000000	0.398482	0.001268	0.081864
max	9599.000000	0.666648	0.010280	0.265956

Congratulations! You've completed this lab. However, you may not notice a green check mark next to this item on Coursera's platform. Please continue your progress regardless of the check mark. Just click on the "save" icon at the top of this notebook to ensure your work has been logged.